Shiv Nadar University Chennai

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

Experiment 1
Implementation of a Single Layer Perceptron for Binary Classification

Objective

The objective of this experiment is to understand the concept of an artificial neuron and implement a Single Layer Perceptron from scratch for binary classification. Students will learn the perceptron learning algorithm, understand the importance of activation functions, visualize the learning process, and evaluate the classifier using a real-world dataset.

Background Theory

An Artificial Neuron is the fundamental building block of a neural network. It receives one or more input features, multiplies them by corresponding weights, adds a bias term, and applies an activation function to determine the output.

The Single Layer Perceptron is the earliest supervised learning algorithm capable of solving linearly separable binary classification problems.

Mathematical Formulation

Weighted Sum

$$z = \mathbf{w}^T\mathbf{x} + b$$

where

- $\mathbf{x}$: Input vector
- $\mathbf{w}$: Weight vector
- b : Bias
- z : Linear combination

Prediction

$$\hat{y} = f(z)$$

Weight Update Rule

$$\mathbf{w}_{new} = \mathbf{w}_{old} + \eta(y - \hat{y})\mathbf{x}$$

Bias Update

$$b_{new} = b_{old} + \eta(y - \hat{y})$$

where η denotes the learning rate.

Activation Functions

An activation function determines the output of a neuron based on the weighted sum of its inputs. It introduces non-linearity, enabling neural networks to learn complex patterns. Although the perceptron uses only the Step Activation Function, modern deep learning models employ differentiable activation functions for efficient optimization through backpropagation.

Step Activation Function

$$f(z) = \begin{cases} 1, & z \geq 0 \\ 0, & z < 0 \end{cases}$$

The Step Activation Function produces a binary output and is therefore suitable only for linearly separable binary classification problems.

Activation	**Output Range**	**Typical Applications**
Step	$\{0, 1\}$	Classical Perceptron
Sigmoid	$(0, 1)$	Binary Classification Output Layer
Tanh	$(-1, 1)$	Hidden Layers (Traditional Networks)
ReLU	$[0, \infty)$	Hidden Layers in CNNs and MLPs
Leaky ReLU	$(-\infty, \infty)$	Avoiding Dying ReLU
Softmax	$(0, 1)$	Multi-class Classification Output Layer

Note: This experiment uses only the Step Activation Function. The remaining activation functions will be studied in subsequent experiments involving Multilayer Perceptrons and Deep Neural Networks.

Dataset Description

Dataset: Banknote Authentication Dataset

Source: UCI Machine Learning Repository

Download Link:

`https://archive.ics.uci.edu/dataset/267/banknote+authentication`

Instances	1372
Features	4 Numerical Features
Classes	2
Missing Values	None
Task	Binary Classification

Feature Description

- Variance
- Skewness
- Curtosis
- Entropy

Target Class

- 0 – Authentic Banknote
- 1 – Forged Banknote

Experimental Procedure

Task 1: Dataset Exploration

- Load the dataset.
- Display the first five samples.
- Determine dataset dimensions.
- Identify missing values.
- Display descriptive statistics.

Expected Output
Dataset summary and statistical information.

Task 2: Exploratory Data Analysis
Generate

- Histograms
- Correlation Heatmap
- Scatter Plot
- Boxplots

Task 3: Data Preprocessing

- Normalize all numerical features.
- Split the dataset into Training (80%) and Testing (20%).

Task 4: Perceptron Implementation
Implement from scratch

- Weight Initialization
- Bias Initialization
- Step Activation Function
- Forward Propagation
- Perceptron Learning Rule

Task 5: Model Training
Display

- Epoch Number
- Misclassified Samples
- Updated Weights
- Updated Bias

Task 6: Model Evaluation
Compute

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix

Source Code

GitHub Repository: `https://github.com/un-shreeya/DeepLearning_Lab`

Experimental Results

Dataset Information

Total Samples	1372
Input Features	4
Target Classes	2 (Fake and Genuine)
Training Samples	1097
Testing Samples	275

Model Performance

Metric	Value
Accuracy	99.27%
Precision	100.00%
Recall	98.43%
F1-Score	99.21%

Final Learned Parameters

Final Weights	[-0.247649, -0.228401, -0.253429, 0.007397]
Final Bias	0.31

Learning Rate Comparison

Learning Rate	Accuracy	Epochs Used
0.001	98.55%	100
0.01	98.91%	100
0.1	98.55%	100

Generated Plots and Interpretation

1. Feature Histograms

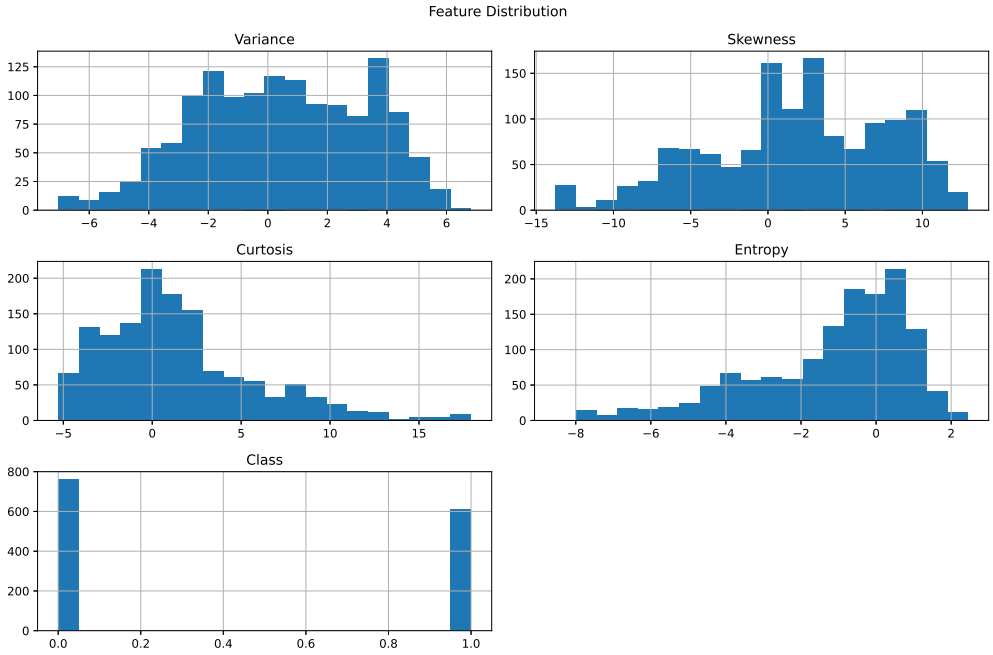

Figure 1: Feature Histograms

Inference: For Variance and Skewness, the distribution seems somewhat balanced. For class, distribution is binary. For Curtosis, it seems skewed towards the right and for Entropy, skewed towards the left.

2. Correlation Heatmap

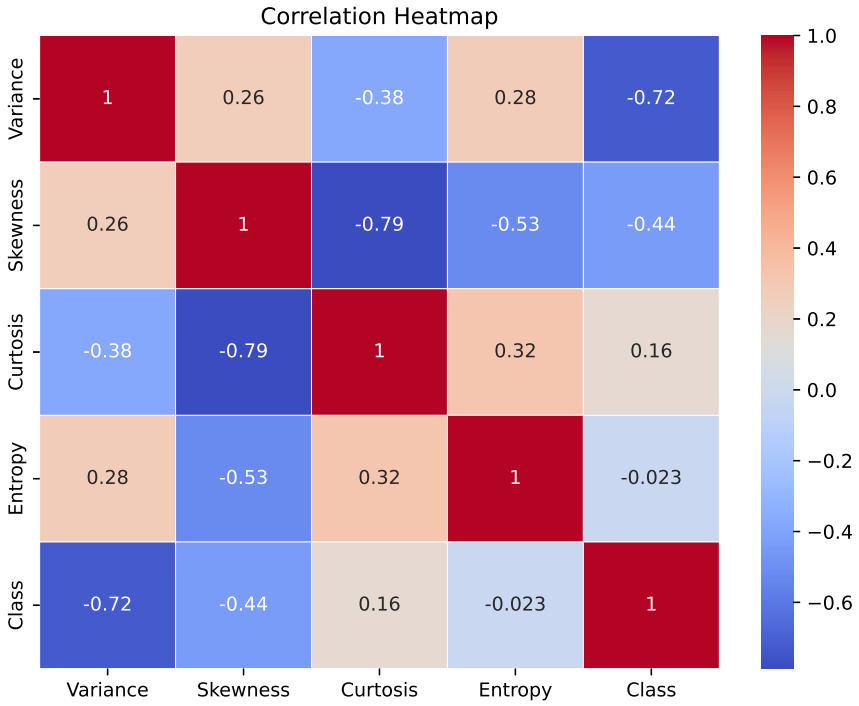

Figure 2: Correlation Heatmap

Inference: For positive numbers (like 0.32), when one variable goes up, the other variable also tends to go up. For negative numbers (like -0.72) when one variable goes up or increases, the other tends to decrease. For numbers close to 0 like -0.023, the variables indicate almost no relation between the 2 variables.

3. Scatter Plots

Variance vs Skewness

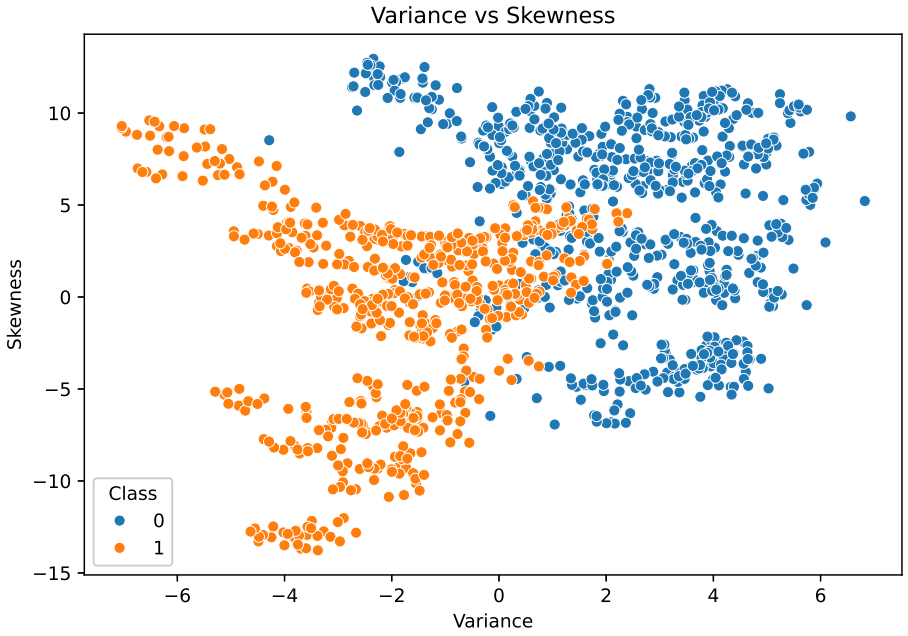

Figure 3: Variance vs Skewness

Inference: If we draw a line from top left to bottom right, we can get most of the data points of blue above and orange below. Its not perfectly separable. We can infer that this is approximately linearly separable.

4. Training Error vs Epoch

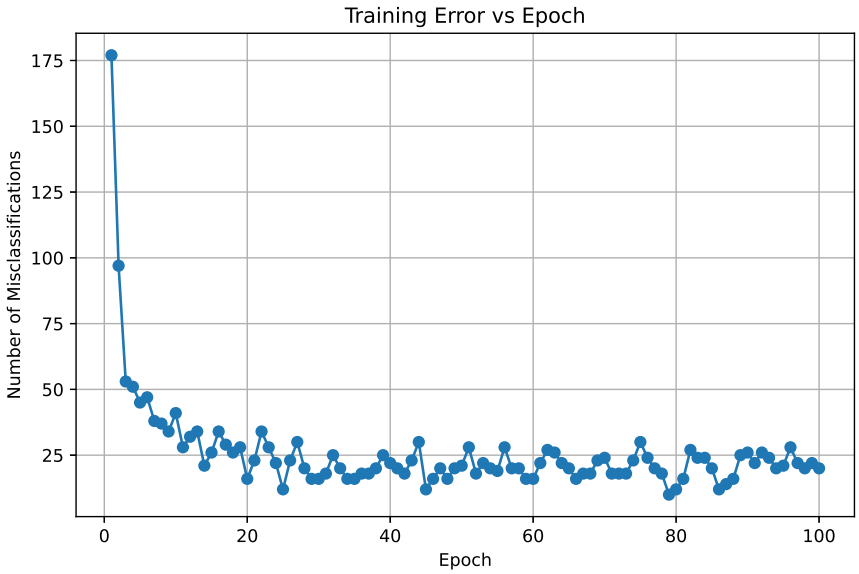

Figure 4: Training Error vs Epoch

Inference: The number of misclassified samples reduces/ seems to stabilise after 20 epochs. It converges here.

5. Weight Evolution

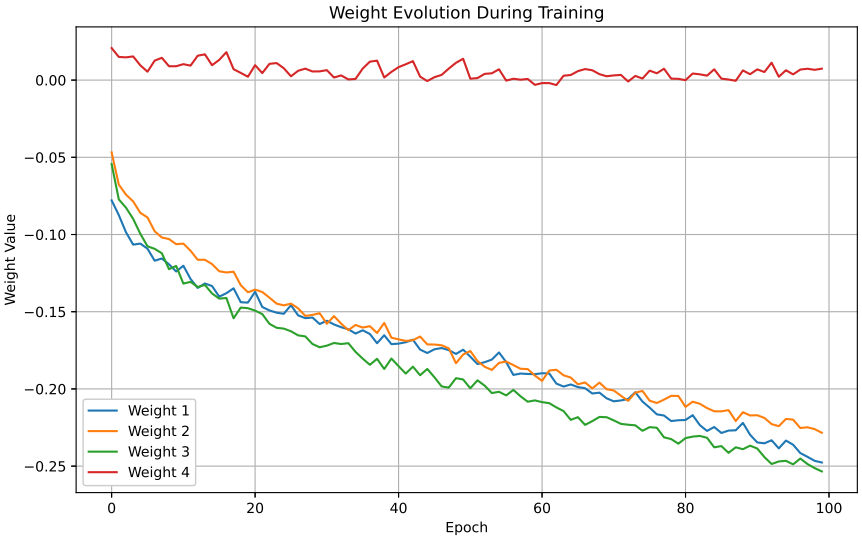

Figure 5: Weight Evolution

Inference: Weights 1,2 and 3 show significant initial adjustments, so that the model can find its optimal decision boundary. Weight 4 remains close to 0 throughout, showing that this feature has

very small influence on the model's decisions.

6. Bias Evolution

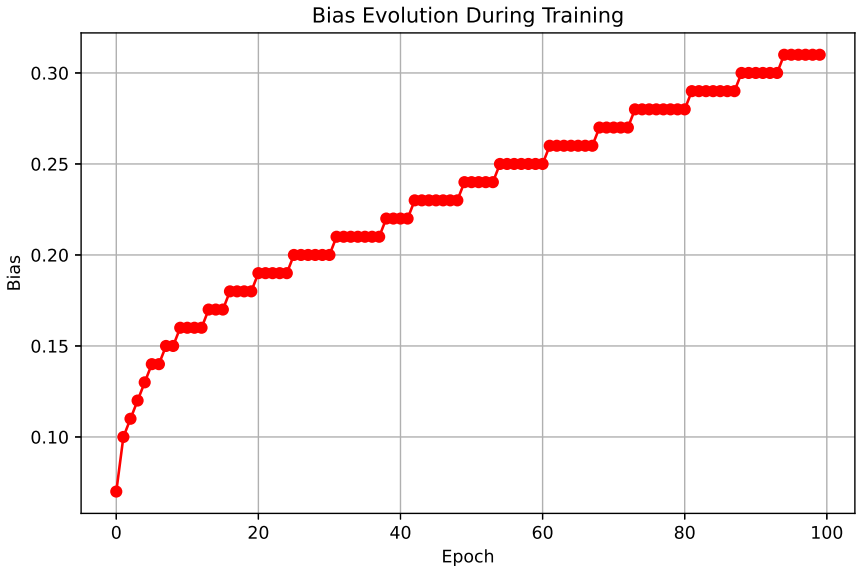

Figure 6: Bias Evolution

Inference: The bias changes steadily over time. The adjustment shows that the model is shifting its decision threshold to better accommodate the class distribution. The step pattern indicates small incremental updates to the bias.

7. Confusion Matrix

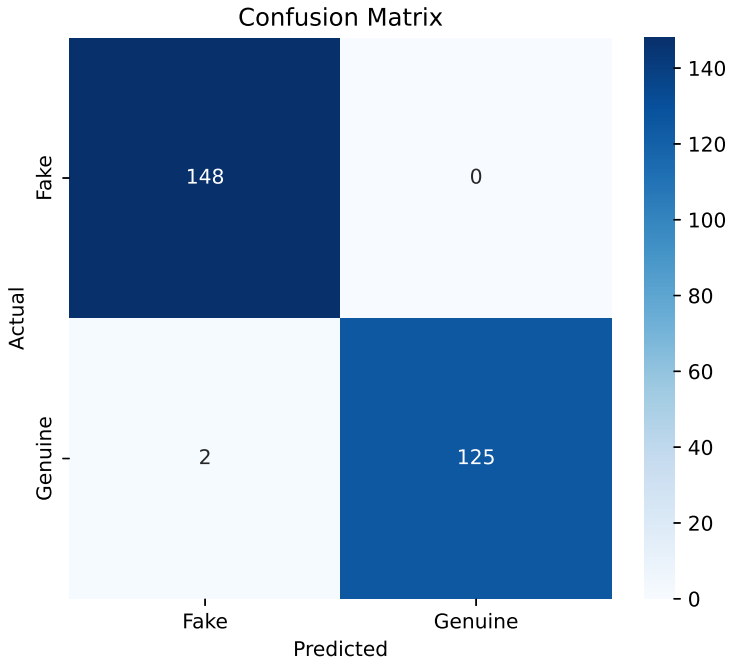

Figure 7: Confusion Matrix

Inference: The confusion matrix shows a large number of True Positives and True Negatives. (148 TN, 125 TP, 0 FP, 2 FN). 273 out of 275 samples have been classified correctly. The perceptron model is highly accurate with minimal error.

8. Learning Rate Comparison

(a) Learning Rate vs Accuracy

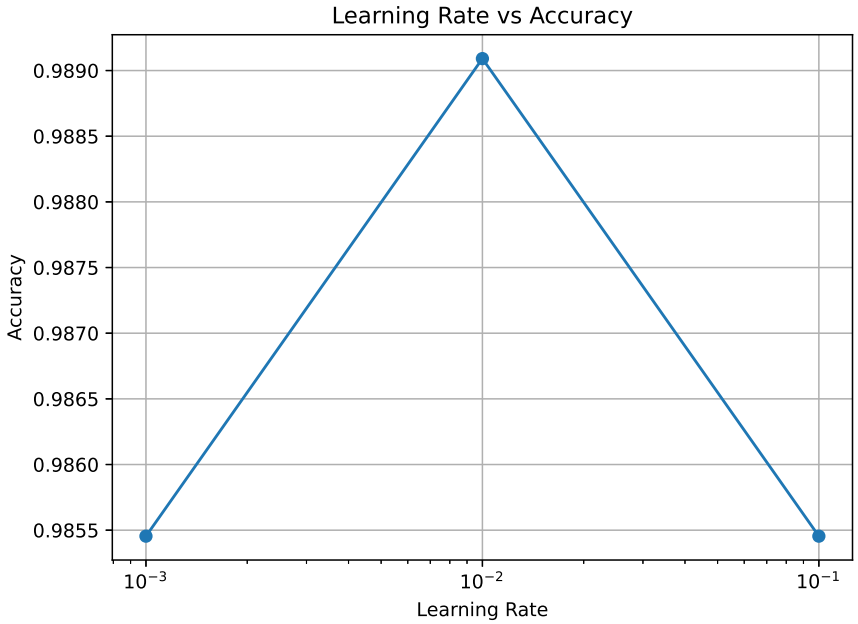

Figure 8: Learning Rate vs Accuracy

Inference: The graph shows a peak at 10^{-2}. This implies that learning rate 0.01 is optimal, resulting in highest accuracy for classification. Both 0.1 and 0.001 learning rates result in reduced performance. This implies that the model is sensitive to this hyperparameter.

(b) Learning Rate vs Epochs Required

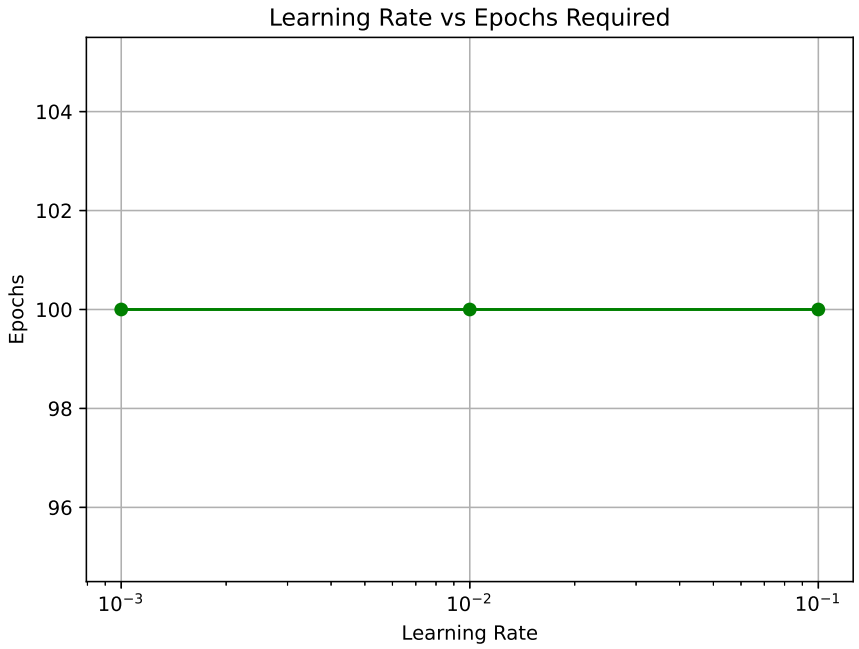

Figure 9: Learning Rate vs Epochs Required

Inference: Learning rate shows no impact on the number of epochs for training. The model reached convergence consistently at 100 epochs across all tested values.

Performance Tables

Training Summary

Dataset Size	1372 samples
Train/Test Split	1097 Training (80%), 275 Testing (20%)
Learning Rate	0.01
Epochs	100
Final Weights	[-0.2476, -0.2284, -0.2534, 0.0074]
Final Bias	0.3100
Accuracy	99.27%
Precision	100.00%
Recall	98.43%
F1-score	99.21%

Epoch-wise Learning

Epoch	Errors	Weight 1	Weight 2	Bias
1	177	-0.07789202	-0.04682787	0.07
50	21	-0.17460017	-0.17773344	0.2400
100	20	-0.24764923	-0.22840107	0.3100

Conclusion:

Conclusion: This activity evaluates the performance and behaviour of a perceptron model. By implementing a Single Layer Perceptron from scratch, this activity provided a hands-on look at how artificial neurons function in binary classification.

References

1. F. Rosenblatt, "The Perceptron," *Psychological Review*, 1958.
2. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016.
3. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.
4. S. Haykin, *Neural Networks and Learning Machines*, Pearson, 2009.
5. UCI Machine Learning Repository – Banknote Authentication Dataset.
6. Scikit-learn Documentation: Perceptron.